BenchGreen Copilot

A scheduling and coordination layer for university wet labs. Takes the week's planned protocols across multiple lab members, finds shared reagents, shared equipment, and compatible waste streams, and returns a coordinated week plan with a concrete environmental impact summary. Hazard and waste classification is grounded in EPA data — not guessed.

The goal is not to redesign experiments. Protocols are sacred. The goal is to stop the same prep, the same gel run, and the same ethanol wash happening three times when it could happen once.

For the hackathon demo we scope to three experiment families that form a connected molecular biology pipeline — DNA extraction → PCR → bead cleanup — with three protocols (different vendors) per family. Nine protocols total. This is deliberate: the three families are sequentially linked in real lab workflows, so the demo tells a coherent story across a full experiment lifecycle.

Built for the SD Hacks 2026 climate/energy/environment track.

Problem

Academic wet labs are shockingly wasteful. A grad student runs PCR with 8 tubes in a 96-well block. The student across the bench preps the same wash buffer separately an hour later. Two centrifuge runs happen underfilled in the same afternoon. Ethidium bromide goes to one waste stream, acrylamide to another, and the person pouring them can't always tell which is which. Reagents expire in freezers because nobody coordinated which day they'd be used.

The waste is environmental (single-use plastic, hazardous chemistry, energy-intensive cold chain) and it's financial (a single-cell run burns $1–3k in kit reagents; if three samples arrive a day apart the lab pays for three runs instead of one).

Existing tools — Labos1point5, My Green Lab's freezer challenge — measure the footprint. None of them coordinate the work.

What BenchGreen does

1. **Takes in a week of planned work** across lab members — who is running what, on which day, with what equipment.
2. **Parses each protocol** into structured steps, reagents, equipment, timing, and waste.
3. **Enriches every reagent** against EPA databases for hazard class, RCRA waste code, and TRI status.
4. **Finds overlaps** — shared reagents, batch-able equipment runs, compatible wash buffers, compatible time windows.
5. **Flags separations** — incompatible waste streams that must stay separate (halogenated vs. aqueous, oxidizers with organics, etc.).
6. **Generates a coordinated schedule** with explanations written in plain English.

7. **Computes impact** — reagent volume saved, plastic saved, kWh saved, hazardous disposal events avoided, annualized if the lab runs the same protocols repeatedly.

The output is a shareable week plan any lab member can open.

Architecture

(See diagram in `/docs/architecture.svg`.)

Three layers:

LLM layer handles the fuzzy work. Protocol parsing — ingesting a paragraph of Word-doc instructions and emitting a structured JSON with reagents, volumes, steps, equipment, and timings. Missing-information detection — when the protocol doesn't say how many samples or doesn't specify the cycler, the LLM flags it and the UI asks the user. Natural-language output — turning the engine's structured recommendations into sentences a grad student will read.

Deterministic engine handles the rigorous work. Matching reagents across protocols. Checking RCRA waste code compatibility. Greedy scheduling: walk the week, sort tasks by priority, merge tasks that share equipment and fall within each other's time windows and have compatible reagents. Pure code, no LLM, fully testable.

EPA enrichment layer is how reagents get their hazard profiles. For each reagent in a parsed protocol, lookup by name or CAS number against:

- **TRI (Toxics Release Inventory)** — is this a reportable chemical, and what waste category.
- **CompTox / IRIS APIs** — hazard classification, cancer/non-cancer endpoints, exposure data. Requires a free API key (email `ccte_api@epa.gov`).
- **RCRA hazardous waste codes** — F-list, P-list, U-list mapping for each reagent. Determines which waste stream a protocol's outputs belong to.

This layer is the reason the tool's outputs are defensible rather than made up.

Data flow, end to end

1. User picks protocols from the seeded library and assigns them to operators and days. (Stretch: PDF upload with LLM parsing.)
2. Each selected protocol is already in our reagent-and-step format (seeded) or gets parsed into that format via LLM.
3. Every reagent is enriched via cached EPA lookups. Hazard class, waste code, and compatibility matrix populated.
4. Engine ingests the week plan plus structured protocols plus lab equipment catalog. Outputs: schedule, coordination recommendations, separation warnings, impact summary.
5. LLM narrator turns the structured output into prose recommendations.

6. UI renders the week view, the recommendation list, and the impact card.

Why this split

The LLM handles what it's good at — natural language, missing-info detection, explanation. The engine handles what the LLM will hallucinate on — "can benzene mix with aqueous waste" is a question you absolutely do not want a language model guessing. Every compatibility call is a table lookup.

Why the EPA enrichment matters

Four distinct uses of hazard classification, in the order the engine uses them:

1. **Waste-stream separation.** Incompatible waste streams cannot merge (halogenated organics vs. aqueous, oxidizers vs. organics, etc.). RCRA codes encode this. Without classification, "don't mix these" is a guess.
2. **Shared-prep validity.** When the engine proposes "prep this reagent once for both tasks," it must check shelf life, storage requirement (flammable cabinet, -20°C, etc.), and reactivity against nearby reagents. EPA hazard flags provide these constraints.
3. **Impact prioritization.** Saving 50 mL of water is worth ~0. Saving 50 mL of phenol-chloroform is high-value because disposal cost and footprint scale with hazard class. Recommendations rank by hazard-weighted savings so the top recommendation is always the highest-impact one.
4. **Citation for demo credibility.** Every hazard call points to an EPA source. "Don't mix these" is a lookup, not an opinion.

In scope (must have)

- 9 seeded protocols across three connected families: DNA extraction (3 vendors), PCR (3 vendors), bead cleanup (3 vendors)
- Reagent term-normalization map (`raw_term → normalized_name → generic_overlap_group`) so cross-vendor reagents that are functionally the same can be identified
- Reagent-and-hazard lookup populated from EPA TRI + CompTox, keyed by `epa_lookup_key` in the reagent map
- Equipment catalog with capacity (thermocycler: 96 wells; centrifuge: 24 slots; magnetic plate: 96 wells; etc.)
- Week-planner UI: pick protocols, assign to people, assign to days
- Overlap engine with: reagent matching (via normalized groups), equipment batching, waste stream separation, greedy scheduler
- Coordination recommendations and separation warnings

- Per-week impact summary (volumes saved, hazardous events avoided, estimated CO_2e range)

In scope — calendar export

- Download week schedule as `.ics` files
- Per-operator `.ics` (each lab member imports their own) and combined lab-wide `.ics`
- Events include the protocol name, equipment assignment as LOCATION, and coordination recommendations in DESCRIPTION

Stretch (nice to have)

- PDF protocol upload and LLM parsing (30-PDF library exists as validation set — goal: parse the 21 not in the seeded 9 and see how many come through cleanly)
- Missing-information interactive resolution (LLM asks clarifying questions)
- Annualized impact projection
- Shareable schedule URL
- Dark mode

Not in scope (explicitly cut)

- User accounts and persistent storage
- AI-generated protocol redesigns
- Waste disposal logistics (which specific hazardous-waste vendor serves the lab)
- Domains beyond molecular biology — no organic synthesis, no cell culture protocols in v1
- Cost accounting in dollars (keep it about environmental footprint; dollar stretch only if time)

Tech stack

Keeping this boring on purpose. We have 1 coder and 2 vibe-coders; we need tools Cursor handles well.

- **Frontend:** Next.js (App Router) + TypeScript + Tailwind. Single app, no separate backend service. Deploy to Vercel.
- **Backend logic:** Next.js API routes in TypeScript. The engine (reagent matching, scheduling) lives in `/lib/engine/` as pure functions — easiest thing for Cursor to help write and for the non-coders to test.
- **LLM:** Gemini API (Google AI) for protocol parsing and recommendation narration. Structured output via JSON mode / function calling.

- **Data:** EPA data (TRI list, RCRA codes, CompTox hazard calls) pre-fetched and cached in a local JSON file at `/data/epa_cache.json` — do not hit EPA live during the demo. Seeded protocols, equipment, operators are JSON files in `/data/seed/`.
- **Storage:** None. Stateless session. All week-plan state lives in browser state (Zustand or React context). Export as JSON for the shareable-schedule stretch.
- **UI components:** shadcn/ui — fast, pretty, Cursor knows it cold.
- **Calendar export:** `ics` npm package.

Why Next.js: single codebase, no CORS, Vercel deploy in 2 minutes, shadcn/ui looks polished without design work. If someone on the team strongly prefers Python, an alternative is Streamlit — uglier but ships faster.

Repo layout

```
benchgreen/
├── app/                      # Next.js pages
|   ├── page.tsx              # Landing + week planner
|   ├── plan/page.tsx         # Planner interface
|   ├── results/page.tsx      # Schedule + recommendations + impact
|   └── api/
|       ├── parse/route.ts    # LLM protocol parsing (stretch)
|       └── plan/route.ts     # Engine endpoint
├── components/               # UI components
|   ├── WeekPlanner.tsx
|   ├── ProtocolCard.tsx
|   ├── ScheduleView.tsx
|   ├── RecommendationList.tsx
|   └── ImpactCard.tsx
├── lib/
|   ├── engine/
|   |   ├── matcher.ts        # reagent + equipment matching
|   |   ├── scheduler.ts      # greedy scheduling
|   |   ├── compatibility.ts  # RCRA / waste-stream rules
|   |   └── impact.ts         # footprint calculations
|   ├── llm/
|   |   ├── parseProtocol.ts
|   |   └── narrate.ts
|   ├── epa/
|   |   └── lookup.ts         # reagent → hazard/waste-code lookup
|   └── export/
|       └── ics.ts            # per-operator and combined .ics generation
├── data/
|   ├── seed/
```

```
|   |   ├── protocols_selected.csv   # 9 protocols (3 families × 3 vendors)
|   |   ├── reagent_term_map.csv     # normalization + hazard flags +
epa_lookup_key
|   |   ├── equipment_term_map.csv   # equipment normalization
|   |   ├── consumables_term_map.csv # tips, tubes, plates, etc.
|   |   ├── waste_rules_map.csv      # waste stream compatibility rules
|   |   ├── overlap_rules.csv        # which generic_overlap_groups can be merged
|   |   ├── protocol_reagents.csv    # per-protocol reagent volumes (to add)
|   |   ├── protocol_thermal_profiles.csv # PCR thermal profiles (to add)
|   |   ├── reagent_stability.csv    # shelf life per generic_overlap_group (to
add)
|   |   ├── equipment.csv            # lab equipment catalog
|   |   └── operators.csv            # example lab members
|   ├── pdf_library/                 # 30 vendor protocol PDFs (validation set)
|   ├── epa_cache.json               # pre-fetched hazard/waste data keyed by
epa_lookup_key
|   └── impact_coefficients.json     # per-reagent footprint estimates
├── docs/
|   ├── architecture.svg
|   └── data_sources.md       # EPA endpoints and how we used them
├── .env.example
└── README.md
```

Data schemas

Seed data lives in CSVs (edited in Google Sheets, exported to `/data/seed/*.csv`). Separate sheets for separate concerns — the engine joins them at runtime. This is the structure the team has already built out:

`protocols_selected.csv`

The 9 seeded protocols across three families. Columns:

- `family` — `DNA_extraction`, `PCR`, or `Bead_cleanup`
- `vendor` — Qiagen, NEB, Thermo Scientific, etc.
- `protocol_name` — full vendor name (e.g. "DNeasy Blood & Tissue")
- `primary_technique` — canonical technique (e.g. `spin_column_silica`, `hot_start_endpoint_PCR`, `SPRI_bead_PCR_cleanup`)
- `why_selected` — free-text justification for inclusion
- `key_overlap_points` — semicolon-separated list of the overlap-relevant reagents/steps ("Proteinase K; Buffer ATL/AL; ethanol-prepped washes; Buffer AE elution")

`reagent_term_map.csv`

The normalization layer. Every reagent that appears across the 9 protocols gets a row. Columns:

- `raw_term` — how it appears in the vendor protocol (e.g. "Buffer AL")
- `normalized_name` — canonical display name
- `generic_overlap_group` — the cross-vendor functional class (e.g. `chaotropic_binding_buffer`, `ethanol_50_to_100`, `low_salt_elution_buffer`) — this is how the engine finds shareable reagents across different-vendor protocols
- `workflow_family` — which family it belongs to
- `stage` — where in the workflow (`lysis`, `bind`, `wash`, `elute`, `setup`, `master_mix`)
- `shareable_prep` — `yes` or `no`; engine's first filter for prep-batching candidates
- `hazard_or_handling_flag` — short flag string (e.g. `chaotropic_salt_bleach_incompatible`, `flammable_solvent`, `ethanol_added_before_use`)
- `epa_lookup_key` — key for the EPA enrichment cache (joins to `epa_cache.json`)
- `protocol_examples` — pipe-separated protocol names where this reagent appears

Sheets to add (gaps from current data)

The current sheets don't yet include these — they need to be added before the engine can run cleanly:

- `protocol_reagents.csv` — per-protocol reagent *volumes*: `protocol_name, reagent_raw_term, volume_per_sample, unit, dead_volume_pct, samples_default, samples_max`. Without this the engine can't compute volume savings or propose shared-prep quantities.
- `protocol_thermal_profiles.csv` — PCR protocols only: `protocol_name, annealing_temp_c, extension_time_s, cycles, denature_temp_c`. Needed for equipment batching (two PCRs can only merge on one block if thermal profile matches).
- `reagent_stability.csv` — per-reagent shelf life after prep: `generic_overlap_group, stable_hours_after_prep, storage_requirement`. Without this, shared-prep recommendations are unsafe ("prep master mix once for Monday and Wednesday" doesn't work if the enzyme is only stable 4 hours).

`equipment.csv` (lab catalog, separate from protocols)

```csv
id,type,model,capacity,block_type,settings_configurable
thermo-c1000-a,thermocycler,Bio-Rad C1000,96,96-well,"annealing_temp,extension_time,
centrifuge-eppendorf-1,microcentrifuge,Eppendorf 5424,24,,
magplate-ambion-1,magnetic_plate,Ambion 96-well,96,96-well,
```

`operators.csv`

```csv
id,name,availability_mon,availability_tue,availability_wed,availability_thu,availabi
op1,Sohini,09:00-17:00,,09:00-17:00,,09:00-13:00
op2,Vikas,,09:00-18:00,09:00-18:00,09:00-18:00,
```

Week plan (user input, not seed)

```json
{
  "week_of": "2026-04-20",
  "tasks": [
    {
      "task_id": "t1",
      "protocol_name": "DNeasy Blood & Tissue",
      "operator": "op1",
      "sample_count": 8,
      "earliest_start": "2026-04-21T09:00",
      "latest_finish": "2026-04-21T17:00"
    }
  ]
}
```

Engine output

```json
```

```
{
  "schedule": [/* tasks with assigned time blocks and equipment */],
  "coordinations": [
    {
      "type": "shared_reagent_prep",
      "overlap_group": "ethanol_50_to_100",
      "tasks": ["t1", "t4"],
      "recommendation": "Prep 30 mL of 70% ethanol once Monday morning; covers both
      "savings": { "volume_ml": 20, "prep_events": 1 }
    }
  ],
  "separations": [
    {
      "type": "waste_stream_conflict",
      "tasks": ["t2", "t7"],
      "reason": "t2 produces chaotropic salt waste (bleach-incompatible); t7 produce
    }
  ],
  "impact": {
    "weekly": {
      "reagent_volume_saved_ml": 45,
      "plastic_items_saved": 18,
      "hazardous_disposal_events_avoided": 2,
      "estimated_co2e_kg_range": [1.2, 2.8]
    },
    "annualized_if_repeated": { /* ...multiplied by frequency */ }
  }
}
```

How the overlap engine reasons (no LLM, no RAG)

The engine is deterministic. These are the four checks, each a table lookup or structured comparison:

Reagent overlap

Build a map: `reagent_name → [(task_id, volume_needed, time_window)]`. Reagents with multiple entries are shared-prep candidates. Filter by: can the combined volume be prepped once within shelf life? Are all tasks within the reagent's usable window after prep? Does storage work (flammable cabinet available, fridge space)? If yes, emit a shared-prep recommendation. Volume saved = sum of per-task dead volumes minus a single prep overhead.

Equipment batching

For each equipment type, get capacity. If two tasks use the same equipment with compatible

settings and overlapping time windows, they can merge into one run. Compatibility is a simple equality check on equipment-relevant protocol fields (e.g. for PCR: `annealing_temp`, `extension_time`, `cycles`). If settings match and combined sample count fits in one block — batch. If settings mismatch — flag the underfilled run as a waste hotspot, no merge.

Waste stream compatibility

For each task, derive its waste streams from its reagents via RCRA code lookup. Pairwise-check task waste streams against a compatibility matrix (hand-built from EPA rules: halogenated never with aqueous, oxidizers never with organics, etc.). If two tasks would share a waste container — check the matrix. Emit separation warnings when required.

Scheduling

Interval scheduling. Tasks have durations, equipment needs, operator windows, and earliest/latest times. Greedy algorithm:

1. Sort tasks by hazard-weighted coordination potential (high-impact tasks first)
2. For each task, assign to the earliest slot where equipment is free, operator is available, and time window is respected
3. When assigning, check if the task enables a coordination with an already-scheduled task — if so, try to align their windows
4. Emit final schedule

Greedy is imperfect but finds the easy wins. Proper CSP solver (OR-Tools) is a day-2 stretch if time allows.

Where the LLM does and doesn't appear. LLM only at the edges: parsing messy input protocols (stretch) and narrating structured recommendations as English output. The middle — matching, compatibility, scheduling — is pure code. No LLM hallucination can touch a safety-relevant decision.

Calendar export

The output week schedule exports as `.ics` files using the `ics` npm package.

- **Per-operator .ics:** Sohini and Vikas each download their own calendar with only their tasks
- **Combined .ics:** whole-lab calendar with operator prefix in event titles

Each event includes:

- `SUMMARY` — protocol name + coordination flag if shared (e.g. "Miniprep (shared ethanol prep with Vikas)")
- `LOCATION` — assigned equipment (e.g. "Thermocycler C1000-A, bench 3")

- `DESCRIPTION` — plain-English recommendations, separation warnings, and prep instructions
- `DTSTART` / `DTEND` — scheduled time block

Users import to Google Calendar, Apple Calendar, or Outlook. Makes the output feel like a real tool, not a demo.

Team split

Three people, ~30 hours. Parallelism or death.

Person A — coder

Owns everything that touches the engine, the LLM integration, and deployment.

- Day 1 hours 0–6: Repo skeleton, Next.js + Tailwind + shadcn/ui scaffolding, CSV loading utility (csv → typed in-memory structures), basic page routing, one working end-to-end stub (pick a protocol → get a trivial hardcoded schedule back).
- Day 1 hours 6–16: Overlap engine. Start with reagent matching via `generic_overlap_group`, then equipment batching, then the greedy scheduler. Unit tests with small synthetic week plans using the 9 seeded protocols.
- Day 2 hours 0–8: Gemini API integration for the narrator (generating plain-English recommendations from the engine's structured output). LLM protocol parser if time. Impact calculator.
- Day 2 hours 8–end: Integration, bug bash, deployment, demo prep.

Lean on Cursor heavily. Write the types first (protocol, reagent-map row, week plan, output) and let Cursor autocomplete the implementations.

Person B — wet-lab #1, domain owner

Owns the substance of the seeded protocols and the reagent-hazard data. This is the highest-leverage role on the team. If the seed data is wrong or thin, the demo lies.

Already done (good work):

- `protocols_selected.csv` with 9 protocols across 3 families (DNA extraction, PCR, bead cleanup)
- `reagent_term_map.csv` with normalization, `generic_overlap_group`, `shareable_prep`, hazard flags, and `epa_lookup_key`
- Supporting sheets: `equipment_term_map`, `consumables_term_map`, `waste_rules_map`, `overlap_rules`
- 30-PDF library archived for LLM-parsing validation (stretch)

To do:

- Day 1 hours 0–6: Add the three missing sheets — `protocol_reagents.csv` (with volumes + sample counts), `protocol_thermal_profiles.csv` (PCR only), and `reagent_stability.csv` (shelf life per overlap group). Without these the engine cannot compute actual savings or propose shared-prep quantities.
- Day 1 hours 6–12: EPA enrichment. Script (with Cursor) lookups for each unique `epa_lookup_key` against TRI and CompTox. Cache results in `/data/epa_cache.json`. Document every assumption in `/docs/data_sources.md` for the reagents that aren't individually tracked by EPA.
- Day 1 hours 12–16: Populate `waste_rules_map.csv` with the actual cross-stream compatibility matrix. Work with Person A to make sure the engine reads it correctly.
- Day 2 hours 0–6: Impact coefficients. For each `generic_overlap_group` and for common plastics, pick a defensible footprint estimate (volume, waste category, rough CO_2e range). Use My Green Lab / published lab carbon-accounting papers. Write into `/data/impact_coefficients.json` with citations.
- Day 2 hours 6–end: Demo script. Write the 90-second pitch. Be the voice on the demo. "I ran this DNeasy extraction six times last week alongside three PCRs and a cleanup; this would have collapsed the ethanol prep and the magnetic plate runs."

Person C — wet-lab #2, UX and polish owner

Owns the UI feel, the demo flow, and the integration glue that vibe-coding handles well.

- Day 1 hours 0–8: shadcn/ui components — week planner view, protocol selection cards (grouped by family: DNA extraction / PCR / bead cleanup), operator picker, day-of-week grid. Cursor does the heavy lifting; focus on layout and information density. Steal from Linear and Cal.com for the week-grid feel.
- Day 1 hours 8–16: Schedule view and recommendation list. This is the money shot of the demo. Every recommendation card needs: the recommendation in plain English, the tasks it combines, the numeric savings, and a "why" tooltip linking to the EPA data that supports the hazard call.
- Day 2 hours 0–6: Impact card. Big number on top (reagent volume saved), smaller supporting numbers, annualized projection with a toggle. Numbers should animate in. This is what judges will remember.
- Day 2 hours 6–end: Landing page, polish pass, dark mode, demo walkthrough practice with Person B.

Shared rituals

- 15-minute standup every 4 hours. What's blocked, what's shipping next.

- One shared Google doc of open questions. If a wet-lab question comes up (can reagent X share a stream with Y?), Person B is the tiebreaker.
- Commit to main at least every 2 hours. Cursor makes it painless. Don't let one person block the others.

Datasets used

- **EPA** (from the non-Scripps allowed list). Specifically:
  - TRI (Toxics Release Inventory) chemical list for reagent hazard status
  - CompTox APIs for hazard class and toxicology endpoints
  - RCRA hazardous waste codes (F/P/U-lists) for waste-stream compatibility
- Citations in `/docs/data_sources.md` with the exact endpoints hit and the dates pulled.

Running locally

```bash
pnpm install
cp .env.example .env.local
# add GEMINI_API_KEY and EPA_CTX_API_KEY
pnpm dev
```

EPA CompTox key: email `ccte_api@epa.gov` for a free key. If you're blocked on that, the cache file at `/data/epa_cache.json` is pre-populated for every reagent in the 9 seeded protocols, so the demo runs without it.

Demo flow (90 seconds)

The demo story is deliberately a connected pipeline: Sohini is extracting DNA, Vikas is PCR'ing it, someone's cleaning up the PCR product. Three stages, same material flowing through.

1. Open the planner. Week is empty.
2. Drop in Sohini's plan: DNeasy Blood & Tissue extraction Monday (8 samples), MagJET DNA extraction Tuesday (4 samples).
3. Drop in Vikas's plan: Q5 Hot Start PCR Tuesday (12 reactions), Platinum II Hot Start PCR Wednesday (8 reactions), Agencourt AMPure XP bead cleanup Thursday.
4. Hit plan. Engine runs. Schedule appears.

5. Top-level result: 4 coordinated changes, 2 separation warnings, 38% reagent volume saved this week.
6. Click the top recommendation: "Prep 60 mL of 70% ethanol once Monday morning. Covers the DNeasy wash (Sohini, Mon), the MagJET wash (Sohini, Tue), and the AMPure cleanup (Vikas, Thu). You'd have prepped three separate times; this consolidates to one." Tooltip shows the `ethanol_50_to_100` overlap group hit across all three protocols, plus EPA classification.
7. Click a separation warning: "Buffer AL (chaotropic, bleach-incompatible) waste must not mix with standard aqueous waste from the PCR cleanup. Keep streams separate." Reason trace links back to the reagent_term_map's `chaotropic_salt_bleach_incompatible` flag.
8. Click the impact card toggle to "annualized." Numbers scale up. "At current protocol frequency, ~2,100 mL reagent saved per year in this lab."
9. Hit "Export calendars." Two `.ics` files download — Sohini's and Vikas's. Open one to show the coordination notes baked into the calendar event description.
10. Hit share. URL copied to clipboard.

That's the pitch. Every claim points to a cite or a calculation. The connected pipeline matters — we're not just finding trivial shared reagents, we're finding shared reagents across different experiment stages that a grad student would genuinely miss.

Known limits (say this out loud in the demo)

- CO_2e numbers are ranges, not precise. Published coefficients for lab reagents vary by an order of magnitude depending on source.
- Our 9 seeded protocols cover three connected families (DNA extraction, PCR, bead cleanup). Other molecular biology workflows (cloning, western blot, cell culture) would need their own reagent maps.
- EPA data classifies commercial chemicals. Some lab buffers are not individually tracked; we bucket them to the nearest reportable component and flag that in the UI.
- Greedy scheduling is a heuristic, not optimal. Good enough for 10-task weeks; we'd need a proper CSP solver at 50+.

Saying these out loud is better than getting asked about them. Judges respect "here's what this tool is not."

After the hackathon (future work slide)

- Lab accounts with persistent protocol libraries
- Freezer/fridge inventory integration (what's already prepped, what's about to expire)

- Full RCRA-aware waste manifest generation for lab safety officers
- Expansion to analytical chemistry and cell culture workflows
- Department-level aggregation: your lab's footprint vs. the building's